

FUN.XYZ · SECURITY REVIEW

Fun.xyz

Vault Wrapper Review.

A security review of Vault Wrapper Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Fun.xyz	Vault Wrapper Review	Guardian	October 10 – October 13, 2025	October 20, 2025

§ 00

Contents.

22 pages — overview, confidence ranking, scope, 12 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 12 findings	P. 08
§ 05	Disclaimer	P. 21
§ 06	About Guardian	P. 22

§ 01

Overview.

Guardian reviewed Vault Wrapper Review across a single review round. The final report includes 5 Low, 7 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	fun-xyz/fun-contracts
CLIENT	METHODOLOGY
Fun.xyz	Smart contract · Manual · Main Review
PROJECT	MAIN REVIEW COMMIT
Vault Wrapper Review	d639a9408e4f
PREPARED BY	
Guardian	
AUDIT WINDOW	
October 10 – October 13, 2025	
FINAL REPORT	
October 20, 2025	

1.1 Confidence Ranking

5 / 5

Very High Confidence

Guardian assigns a Confidence Ranking of 5 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	0	0	0	0	0
Medium	0	0	0	0	0
Low	5	0	5	0	0
Informational	7	0	7	0	0
All severities	12	0	12	0	0

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence ASSIGNED	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Vault Wrapper Review in `fun-xyz/fun-contracts`. The scope includes 252 source lines across 9 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
src/VaultWrapper.sol	48	155
src/Utils/Chains.sol	88	140
src/Utils/Replacer.sol	26	66
src/Utils/VaultDepositManager.sol	39	127
src/Utils/VaultWrapperErrors.sol	14	45
src/Utils/VaultWrapperEvents.sol	6	27
src/Utils/WETHHelper.sol	15	43
src/CREATE3/CREATE3Factory.sol	13	26
src/CREATE3/ICREATE3Factory.sol	3	14

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
L-01	Lacking VaultABI Restriction	Validation	LOW	ACKNOWLEDGED	p. 8
L-02	Functions May Not Be Native Compatible	Warning	LOW	ACKNOWLEDGED	p. 9
L-03	Native Funds Trapped If allowFailure True	Warning	LOW	ACKNOWLEDGED	p. 10
L-04	Mint Actions Do Not Refund The User	Warning	LOW	ACKNOWLEDGED	p. 11
L-05	Replacer Incompatible With Dynamic Types	Logical Error	LOW	ACKNOWLEDGED	p. 12
I-01	Unnecessary Ownership SSTORE	Gas Optimization	INFORMATIONAL	ACKNOWLEDGED	p. 14
I-02	Missing Update Configuration	Configuration	INFORMATIONAL	ACKNOWLEDGED	p. 15
I-03	Missing Event Data	Events	INFORMATIONAL	ACKNOWLEDGED	p. 16
I-04	Native Functionality Can Be Incompatible	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 17
I-05	Tokens Exceeding 18 Decimals Incompatible	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 18
I-06	Lacking Vault Code Existence Check	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 19
I-07	Fee On Transfer Tokens Unsupported	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 20

LOW **L-01**

ACKNOWLEDGED

Lacking VaultABI Restriction

CATEGORY	Validation
LOCATION	VaultDepositManager.sol
REVIEW	Main Review

— DESCRIPTION

In the `registerVault` function, the `vaultABI` parameter is provided to represent the target function that is callable on the vault.

However this `vaultABI` parameter is only emitted in the `VaultRegistered` event and is not used in the logic or storage resulting from the `registerVault` function.

When a vault is configured to be used with the `VaultWrapper` system any function on it may be invoked through the `VaultWrapper`, even those that are not intending to be called through the expected execution flow.

Given that the `VaultWrapper` is intended to work with arbitrary vault implementations, it would be prudent to limit the callable function signatures to the expected set for a vault.

— RECOMMENDATION

It may be expected that the `vaultABI` string is only emitted in the `VaultRegistered` event. However it would be prudent to consider storing a set of whitelisted `bytes4` function selectors for each vault to explicitly define the expected interaction with a supported vault.

— RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Functions May Not Be Native Compatible

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

The `VaultDepositManager` assigns a `FLAG_ALLOW_NATIVE` flag for the entire vault, which may have multiple functions that are immediately supported when a vault is registered.

However not all of these functions that are now callable on a vault may support native payment. For example, often vaults have a `deposit` and a `depositNative` type function, where the `deposit` function would not be compatible with native Ether.

— RECOMMENDATION

Be aware of this generalization that the configuration makes. The native support could be indicated for each supported selector for a vault, however this may introduce more complexity than such a validation is worth.

— RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Native Funds Trapped If allowFailure True

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

The expected flow for any vault functions that accept native payment is to include a direct Ether transfer to the `VaultWrapper` contract in the multicall before the relevant deposit action.

Relay protocol allows calls to be configured with a `allowFailure` value, where when true, if the call fails as a part of the multicall, it continues with execution of the batch.

However in the case of the `VaultWrapper`'s native use-case this could result in native funds being trapped if `allowFailure` is used for the invocation of the `VaultWrapper.deposit` function which then fails for any reason at the Fun wrapper level or at the underlying vault level.

RECOMMENDATION

In the offchain UI configuration of these actions do not allow `allowFailure` to be configured as true. Additionally, consider documenting this risk for integrators.

RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

Mint Actions Do Not Refund The User

CATEGORY	Warning
LOCATION	VaultWrapper.sol
REVIEW	Main Review

— DESCRIPTION

The mint function of the ERC4626 standard will not always transfer the entire input amount from the depositor, but instead the amount that is necessary to mint the specified shares.

The `VaultWrapper` system should be considered incompatible with any `mint` or `mint-like` functions which do not take an explicit deposit input amount since they will leave input tokens in the `VaultWrapper` contract which are susceptible to being stolen by the next depositor.

— RECOMMENDATION

Be aware and clearly document that any `mint` function or function that does not accept and use an explicit input token amount is strictly incompatible with the `VaultWrapper` system.

— RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

Replacer Incompatible With Dynamic Types

CATEGORY	Logical Error
LOCATION	Replacer.sol
REVIEW	Main Review

— DESCRIPTION

The replacer always assumes that values are left padded to fit a 32 byte increment and that their value will be interpreted using the whole 32 byte word (as is the case with static types such as `address` `uint` etc...).

However this is often not the case with dynamic types which will not be left padded and are instead interpreted using their length entry.

For example, consider the following function calldata for a function `deposit(bytes amount)`, where the amount is encoded in a bytes object and those bytes are expected to be a uint64 (less than a full word).

```
bytes memory amount = hex"FF"; // Assume amount is represented by the 1 byte 0xFF for demonstration
```

```
└ Hex (Memory):
```

[illegible][illegible]

```
→ abi.encodeWithSignature("deposit(bytes)", amount)
```

```
Type: dynamic bytes
```

└ Hex (Memory):

[illegible]

```
└─ Contents ([0x20:..]):
```

[illegible]

The calldata is:

[illegible]

As you can see the ff value is left aligned rather than right aligned. If this amount

L-05 Replacer Incompatible With Dynamic Types**CONTINUED**

value is replaced with the placeholder, the replace function will replace the bytes assuming that the value is right aligned and uses the entire word. Which will provide an invalid amount of 0 given the length value will not reach a right aligned value.`

— RECOMMENDATION

Be aware that the replacer is not compatible with any dynamic type encodings. These should be rare in any vault implementation and therefore are not worth supporting given the complexity. The `VaultWrapper` should only be used with vaults that use a static type encoding for fields that ought to be replaced.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-01

ACKNOWLEDGED

Unnecessary Ownership SSTORE

CATEGORY Gas Optimization
LOCATION VaultDepositManager.sol
REVIEW Main Review

DESCRIPTION

Upon production deployment of the `VaultWrapper`, the owner is transferred twice in the same deployment transaction when the initial owner is a multisig and will not be the deployer.

This is to support the logic where if the `initialOwner` address is zero then the owner can default to the `msg.sender`. However a more efficient way to achieve this is to simply use a ternary case in the constructor invocation of the `Ownable` constructor.

RECOMMENDATION

In the constructor of the `VaultDepositManager` contract, consider removing the `initialOwner != address(0)` case and instead passing the initial owner address as such to the `Ownable` constructor:

```
constructor(address initialOwner) Ownable(initialOwner == address(0) ? msg.sender :  
initialOwner)
```

This way avoiding duplicating `_transferOwnership` calls.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

Missing Update Configuration

CATEGORY	Configuration
LOCATION	VaultDepositManager.sol
REVIEW	Main Review

DESCRIPTION

In the `VaultDepositManager` contract the only function which exposes configuration for the `vaultConfigs` mapping is the `registerVault` function.

However this function can only be called once, when the vault is initially being configured and not again until the vault has been removed.

It may however be useful to update vault configuration if for example later it is desired to support native functions with a vault.

RECOMMENDATION

Be aware that a vault will have to be removed to be reconfigured. Consider adding an update configuration function in case it is needed.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-03**

ACKNOWLEDGED

Missing Event Data

CATEGORY	Events
LOCATION	VaultDepositManager.sol
REVIEW	Main Review

DESCRIPTION

The `VaultRegistered` event includes all of the details around the configuration that was made for the corresponding vault except the `allowsNative` value that was configured.

RECOMMENDATION

Consider including the `allowsNative` value in the `VaultRegistered` event that is emitted.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-04**

ACKNOWLEDGED

Native Functionality Can Be Incompatible

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

For native value deposits the `VaultWrapper` system expects that the Relay router has transferred the native funds to the `VaultWrapper` contract ahead of the `deposit` invocation.

The only compatible way to do this for e.g. a swap is if the swap uses the `VaultWrapper` as the output address. Otherwise if the relay router receives the output tokens, like in the [example transaction provided] (<https://hyperevmscan.io/tx/0x33d309fa50e8db606e951872389b89ab8a579c1067ca8e3e340b17e4b94f7bac>) where USDC is swapped for USDT, then there is no way to be sure you can transfer all Ether to the `VaultWrapper` contract since the ether amount would have to be encoded in one of the call objects ahead of time.

For some esoteric systems, it may not be possible to specify a receiver and instead the native output may be forced to go to the Relay router address thus breaking the use-case of the fun `VaultWrapper` contract.

— RECOMMENDATION

Simply be aware of this limitation of support for Native deposits and consider it when evaluating which swap venues to use.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-05**

ACKNOWLEDGED

Tokens Exceeding 18 Decimals Incompatible

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

When a vault is configured with the `FLAG_CONVERT_TO_18_DECIMALS` flag, the tokens are converted to 18 decimals for the deposit calldata. However only tokens that have less than 18 decimals are converted up to 18.

Tokens with larger than 18 decimals, such as YAM v2 which have 24 decimals will not be converted to 18 decimals. For any vault that requires 18 decimal representations this will break the integration.

RECOMMENDATION

Consider explicitly disallowing tokens that have larger than 18 decimals in the `FLAG_CONVERT_TO_18_DECIMALS` handling.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-06**

ACKNOWLEDGED

Lacking Vault Code Existence Check

CATEGORY	Validation
LOCATION	VaultDepositManager.sol
REVIEW	Main Review

DESCRIPTION

In the `registerVault` function there is no validation that the vault contract houses bytecode. As a result, when a deposit is executed, the `.call` will return true even though no deposit was made. And the transaction will silently complete while leaving user tokens in the `VaultWrapper` contract.

RECOMMENDATION

Consider validating that the byte code of the vault address is nonzero when registering to avoid any mistakes that can cause loss of funds.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-07**

ACKNOWLEDGED

Fee On Transfer Tokens Unsupported

CATEGORY	Warning
LOCATION	VaultWrapper.sol
REVIEW	Main Review

DESCRIPTION

The contract compares `minAmountOut` to `IERC20(token).balanceOf(msg.sender)` before transfer. For fee-on-transfer or negatively rebasing tokens, the wrapper will receive fewer tokens than the sender's reported balance, but the deposit still proceeds, undermining slippage protection. The vault may then deposit a smaller net amount than the user intended.

RECOMMENDATION

Be aware that fee on transfer tokens are not supported with the `VaultWrapper`.

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.